Advanced Greedy

Exchange arguments, scheduling, and greedy with regret

Learning goals

- Prove a sorting greedy with an **adjacent exchange argument**
- Derive the comparator from the proof (and make it exact)
- Solve deadline scheduling with a heap
- Use **regret greedy**: take now, undo the worst choice later
- Know when greedy is wrong and DP is needed

Why greedy is dangerous at regionals

- Greedy solutions are short → tempting → many WAs
- Medium regional problems are often "greedy **with a proof**"
- A 3-minute exchange argument on paper beats two 20-minute penalties
- Rule: **no proof sketch, no submission** (or a brute-force stress test)

The exchange argument template

1. Take any optimal order; suppose it has adjacent `i, j` in the "wrong" order
2. Swapping `i, j` affects only their own terms
3. Show the swap never makes the answer worse
4. Repeat → the sorted order is optimal

> The inequality you derive in step 3 **is** your comparator.

Example 1: weighted completion time

- n jobs, job i takes `t_i`, weight `w_i`; minimise Σ `w_i · C_i`
- Adjacent `i` then `j`, starting at time T:

```
i first:  w_i(T+t_i) + w_j(T+t_i+t_j)
j first:  w_j(T+t_j) + w_i(T+t_j+t_i)
difference (i first − j first) = w_j·t_i − w_i·t_j
```

- `i` first is no worse iff `t_i · w_j ≤ t_j · w_i` (Smith's rule)

Comparator: exact, not floating point

```
sort(job.begin(), job.end(), [](const auto& a, const auto& b) {
    // a = (t, w); a before b iff t_a * w_b < t_b * w_a
    return a.first * b.second < b.first * a.second;
});
long long time = 0, cost = 0;
for (auto [t, w] : job) { time += t; cost += w * time; }
```

- Cross-multiply instead of comparing `t/w` doubles
- Use `<`, never `<=` (strict weak ordering, or `sort` may crash)
- Products up to 10^{18}? Check fits in `long long` or use `__int128`

Example 2: projects with rating changes

Codeforces 1203F1 — Complete the Projects: rating `r`; project needs `r ≥ a_i`, then `r += b_i`; must stay ≥ 0. Can you do all?

- Gains (`b ≥ 0`): do them first, sorted by `a` ascending
- Losses (`b < 0`): adjacent `i, j` feasible as `i, j` needs `r ≥ max(a_i, a_j − b_i)`; exchange shows **sort by `a + b` descending**
- Real lesson: split into classes, then exchange within each class

Example 3: stacking (Cow Acrobats)

- Each cow: weight `w`, strength `s`; risk = (weight above) − `s`
- Minimise the maximum risk of a tower
- Exchange two adjacent cows: the pair's max risk is smaller when the one with smaller `w + s` is on top
- Sort by `w + s` ascending from the top

POJ 3045 — Cow Acrobats

Deadlines: what kind of problem is it?

Variant	Method
Unit jobs, profits, deadlines	sort by deadline + min-heap of profits, or scan time backwards with max-heap
Jobs with durations, maximise count on time	**Moore–Hodgson** (max-heap of durations)
Reward `d − f` for each job	sort by duration (CSES 1630)
Durations + profits, maximise profit	knapsack-like DP sorted by deadline

Scan backwards: Summer Vacation

AtCoder ABC 137 D: job pays `B_i` if done ≥ `A_i` days before the end.

```
days left:  k = 1, 2, ..., M
jobs usable with k days left:   A_i ≤ k   (a growing set!)
→ push them into a max-heap, take the best one
```

- Backwards, the set of available jobs only **grows** → heap works
- Same structure: Kattis `bank` (Bank Queue)

Moore–Hodgson

- Maximise number of jobs finished by their deadlines
- Process by deadline; keep chosen jobs in a max-heap by duration
- If the schedule becomes late: remove the **longest** chosen job

```
sort(jobs.begin(), jobs.end());          // (deadline, duration)
for (auto [d, p] : jobs) {
    chosen.push(p); total += p;
    if (total > d) { total -= chosen.top(); chosen.pop(); }
}
// answer = chosen.size()
```

Why Moore–Hodgson works

- Invariant: after processing jobs with deadline ≤ D, the heap holds a **maximum-size** feasible set of them with **minimum total duration**
- Adding one job can increase the count by at most 1
- If infeasible, dropping the longest keeps the count and minimises the total
- Minimum total leaves the most room for future jobs

Regret greedy: the idea

```
take every option greedily
  └─ if a constraint breaks:
       undo the worst decision made so far
       (the heap remembers all decisions)
```

- Correct when "undo the worst" provably restores optimality
- Signals: "choose a subset in order", "prefix constraint", "buy/sell"
- Always stress test against a brute force

Regret: Potions (CF 1526C2)

- Drink potions left to right, health starts at 0, must stay ≥ 0; maximise count

```
priority_queue<ll, vector<ll>, greater<ll>> taken;   // min-heap
ll health = 0;
for (ll a : potions) {
    taken.push(a); health += a;
    if (health < 0) { health -= taken.top(); taken.pop(); }
}
// answer = taken.size()
```

- Removing the most negative taken potion: count stays, health is maximal

Regret: Buy Low Sell High (CF 865D)

- Each day buy one share, sell one, or nothing; maximise profit

```
for (ll p : price) {
    if (!pq.empty() && pq.top() < p) {
        profit += p - pq.top(); pq.pop();
        pq.push(p);                  // "undo": re-sell later at a higher price
    }
    pq.push(p);                      // "buy today" option
}
```

- Selling at `p` then later at `q > p`: pushing `p` again lets us replace the sale, gaining `q − p`

Merging greedy (Huffman)

- Cut a stick into given pieces; cost of a cut = current length (CSES 1161 *Stick Divisions*)
- Reverse it: **merge** pieces; always merge the two smallest

```
priority_queue<ll, vector<ll>, greater<ll>> pq(a.begin(), a.end());
ll cost = 0;
while (pq.size() > 1) {
    ll x = pq.top(); pq.pop(); ll y = pq.top(); pq.pop();
    cost += x + y; pq.push(x + y);
}
```

Marginal gains with a heap

- "Split k resources among n items; each extra unit has decreasing benefit"
- Push the gain of the next unit for each item; repeatedly take the best
- Correct when gains are **concave** (non-increasing per item)
- Example: Codeforces 1428E *Carrots for Rabbits*

When greedy fails

- Coin change with arbitrary denominations: {1, 3, 4}, target 6 → greedy 4+1+1, optimal 3+3
- Knapsack 0/1: best ratio first is wrong
- Interval scheduling **with weights**: need DP + binary search
- Test: try to break your greedy with 3–4 element cases before coding

Proof or stress test

```
greedy idea ──▶ 3-element exchange check on paper
            ──▶ brute force over all permutations / subsets (n ≤ 8)
            ──▶ random tests, 1000 cases
```

- Every solution in today's notes was checked this way
- A brute force for a scheduling problem is ~10 lines with `next_permutation`

Complexity

Technique	Time
Sort with exchange comparator	O(n log n)
Moore–Hodgson	O(n log n)
Backward scan with heap	O((n + M) log n)
Regret greedy	O(n log n)
Huffman merging	O(n log n)

Common pitfalls

- Comparator with `<=` or non-transitive logic → undefined behaviour
- Floating-point ratios in comparators → ties broken randomly
- Overflow in cross-multiplication
- Forgetting a class split (gains vs. losses)
- Regret greedy without proof → plausible but wrong

Practice problems

- CSES 1630 — Tasks and Deadlines; CSES 1161 — Stick Divisions
- CSES 1632 — Movie Festival II
- AtCoder ABC 137 D — Summer Vacation
- Kattis `bank` — Bank Queue
- Codeforces 1203F1 — Complete the Projects (easy)
- Codeforces 1526C2 — Potions (Hard Version)
- Codeforces 865D — Buy Low Sell High
- Codeforces 1428E — Carrots for Rabbits
- POJ 3045 — Cow Acrobats

Summary

- Exchange argument ⇒ comparator; compare exactly with integers
- Deadline problems: sort by deadline, fix violations with a heap
- Regret greedy: commit, then undo the worst choice when forced
- Always proof-sketch or stress test before submitting
- Next: graph modeling — when the graph is not given